



ЗВІТ

Лабораторна робота № 4

Дисципліна: Базы даних та інформаційні системи

**Тема: Розширення можливостей PostgreSQL: користувацькі типи,
функції та тригери**

Виконав

Студент 3 курсу

Групи МІТ-31

Факультету інформаційних технологій

Бабяк Володимир

Мета роботи :

Закріпити знання з розширюваності PostgreSQL. Навчитися створювати користувацькі типи даних. Реалізувати власну користувацьку функцію або агрегат. Створити тригери для логування змін у базі даних. Автоматично оновлювати пов'язані таблиці чи заповнювати значення. Оновити діаграму бази даних відповідно до виконаних завдань. Перевірити коректність роботи реалізованих об'єктів через виконання тестових SQL-запитів.

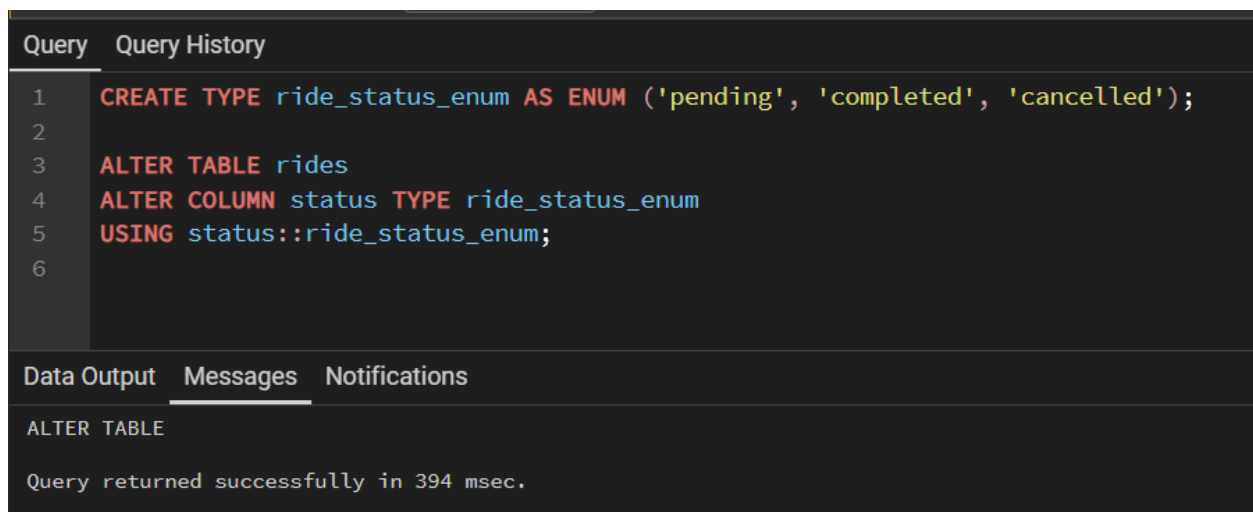
Хід роботи

Варіант 19 – База даних для служби таксі

Опис внесених змін та SQL-коди реалізації:

1. Створення користувацького типу даних

Для забезпечення цілісності даних статус поїздки (очікується, завершено, скасовано) винесено в окремий тип ENUM. Це унеможливило введення некоректних статусів у таблицю rides.



```
Query  Query History
1  CREATE TYPE ride_status_enum AS ENUM ('pending', 'completed', 'cancelled');
2
3  ALTER TABLE rides
4  ALTER COLUMN status TYPE ride_status_enum
5  USING status::ride_status_enum;
6

Data Output  Messages  Notifications
ALTER TABLE
Query returned successfully in 394 msec.
```

2. Створення користувацької функції

Для спрощення аналітики та винесення логіки на рівень бази даних було створено функцію, яка автоматично вираховує загальний дохід конкретного водія за всі його успішно завершені поїздки.

```
1 CREATE OR REPLACE FUNCTION get_driver_revenue(drv_id INT)
2 RETURNS NUMERIC AS $$
3 BEGIN
4     RETURN (
5         SELECT SUM(price)
6         FROM rides
7         WHERE driver_id = drv_id AND status = 'completed'
8     );
9 END;
10 $$ LANGUAGE plpgsql;
11
12 -- Викликаємо функцію для перевірки:
13 SELECT get_driver_revenue(1) AS total_revenue;
14
```

Data OutputMessagesNotifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	total_revenue
1	980.00

Демонстрація роботи функції підрахунку доходу. База даних автоматично просумувала вартість усіх успішних поїздок для водія з ID 1 і повернула підсумкове число.

3. Створення тригера для логування змін

Для аудиту бази даних реалізовано механізм логування. Було створено таблицю `rides_log` та тригер, який автоматично фіксує кожну зміну статусу поїздки (наприклад, коли водій бере замовлення або скасовує його).

Query	Query History
1	-- Таблиця для логів
2	CREATE TABLE rides_log (
3	log_id SERIAL PRIMARY KEY,
4	ride_id INT,
5	operation VARCHAR(10),
6	changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7	);
8	
9	-- Функція тригера
10	CREATE OR REPLACE FUNCTION log_ride_changes() RETURNS TRIGGER AS \$\$
11	BEGIN
12	IF (TG_OP = 'DELETE') THEN
13	INSERT INTO rides_log (ride_id, operation) VALUES (OLD.id, 'DELETE');
14	RETURN OLD;
15	ELSIF (TG_OP = 'UPDATE') THEN
16	INSERT INTO rides_log (ride_id, operation) VALUES (NEW.id, 'UPDATE');
17	RETURN NEW;
18	ELSIF (TG_OP = 'INSERT') THEN
19	INSERT INTO rides_log (ride_id, operation) VALUES (NEW.id, 'INSERT');
20	RETURN NEW;
21	END IF;
22	RETURN NULL;
23	END;
24	\$\$ LANGUAGE plpgsql;
25	
26	-- Прив'язка тригера
27	CREATE TRIGGER trg_rides_audit
28	AFTER INSERT OR UPDATE OR DELETE ON rides
29	FOR EACH ROW EXECUTE FUNCTION log_ride_changes();

Перевірка нових статусів (ENUM)

```
1 SELECT ride_id, client_id, driver_id, status FROM rides;
```

	ride_id [PK] integer	client_id integer	driver_id integer	status ride_status_enum
1	1	1	1	completed
2	2	2	2	completed
3	3	3	1	completed
4	4	4	3	completed
5	5	5	2	cancelled
6	6	1	3	completed
7	7	2	1	completed
8	8	3	2	completed
9	9	4	1	completed
10	10	5	3	completed

Перевірка функції доходу водія

```
1 SELECT get_driver_revenue(1) AS total_revenue;
```

	total_revenue numeric
1	980.00

Перевірка тригера логування

1

UPDATE rides SET status = 'completed' WHERE ride_id = 1;

2

3

SELECT * FROM rides_log;

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

	log_id [PK] integer	ride_id integer	operation character varying (10)	changed_at timestamp without time zone
1	1	1	UPDATE	2026-05-14 10:57:35.899959

Перевірка автоматичного оновлення

Query

Query History

1

INSERT INTO rides (client_id, driver_id, distance_km, price, status)

2

VALUES (1, 1, 12.5, 300.00, 'pending');

3

4

SELECT * FROM clients;

Data Output

Messages

Notifications

≡+

▼

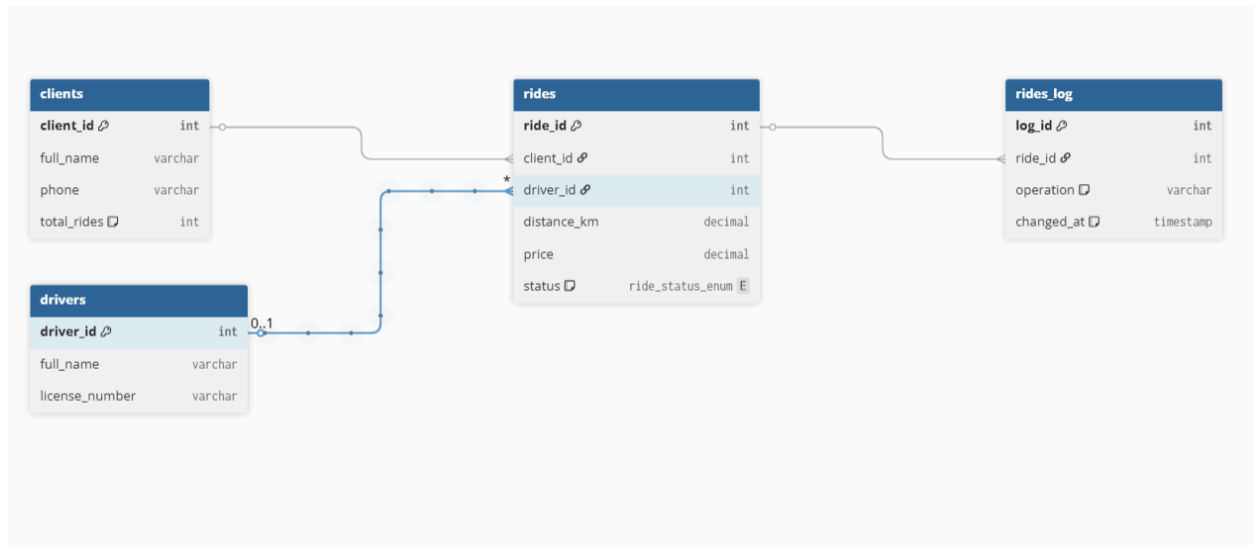
▼

SQL

	client_id [PK] integer	first_name character varying (50)	last_name character varying (50)	phone_number character varying (20)	total_rides integer
1	2	Марія	Шевченко	+380671234567	0
2	3	Іван	Бойко	+380631234567	0
3	4	Олена	Ткаченко	+380991234567	0
4	5	Дмитро	Кравченко	+380971234567	0
5	1	Олександр	Коваленко	+380501234567	1

4. Оновлена ER-діаграма бази даних

У структуру бази даних було додано нову таблицю rides_log для збереження історії зміни статусів поїздок. Вона пов'язана з таблицею rides за атрибутом ride_id. Також оновлено тип даних колонки status на створений ride_status_enum.



5. Сценарії подальшого розширення

- Автоматичне оновлення рейтингу водія в таблиці `drivers` після кожної успішної поїздки.
- Функція динамічного ціноутворення, яка прийматиме відстань та поточний час (нічний тариф) і автоматично розраховуватиме значення поля `price` перед вставкою нового запису в таблицю `rides`.

6. Висновки щодо ефективності реалізованих рішень

У результаті виконання лабораторної роботи №4 розширено функціонал бази даних служби таксі.

- Застосування ENUM для статусів поїздок підвищило цілісність даних і унеможливило помилки при ручному введенні.
- Написання користувацької функції `get_driver_revenue` дозволило винести агрегаційну логіку (підрахунок заробітку) на рівень СУБД, що значно прискорює та спрощує аналітичні запити з боку клієнтських додатків.
- Реалізація тригера та логуючої таблиці `rides_log` забезпечила надійний автоматизований аудит системи, що дозволяє відстежувати життєвий цикл кожного замовлення без необхідності втручання зовнішнього коду.